

Power Calculation: Normal Method vs Fast Method

Objective: To calculate x raised to the power n and compare the normal multiplication method with the fast exponentiation method.

1. Normal Method of Power Calculation

In the normal method, the base x is multiplied by itself repeatedly n times. For example, 2^5 is calculated as $2 \times 2 \times 2 \times 2 \times 2$. The number of operations increases directly with the exponent, so the time complexity is $O(n)$.

2. Fast Method - Exponentiation by Squaring

The given C program uses **fast exponentiation**. It recursively divides the exponent by 2 and calculates the half-power only once. For an even exponent it returns **half x half**, while for an odd exponent it returns **$x \times$ half x half**. Since the exponent is halved at every recursive step, its time complexity is $O(\log n)$.

3. C Code - Fast Power Method

```
#include <stdio.h>

long long powerFast(int x, int n)
{
    if (n == 0)
        return 1;

    long long half = powerFast(x, n / 2);

    if (n % 2 == 0)
        return half * half;
    else
        return x * half * half;
}

int main()
{
    int x, n;

    printf("Enter base: ");
    scanf("%d", &x);

    printf("Enter exponent: ");
    scanf("%d", &n);

    printf("%d^%d = %lld\n", x, n, powerFast(x, n));

    return 0;
}
```

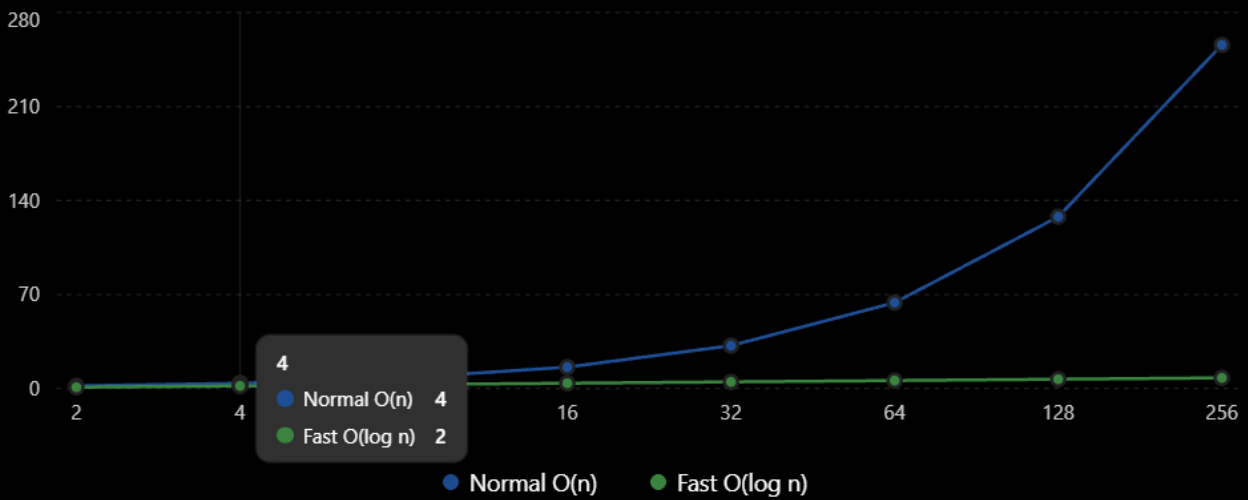
4. Comparison

Method	Approach	Time Complexity
Normal Power	Multiply x repeatedly n times	$O(n)$
Fast Power	Divide exponent by 2 recursively	$O(\log n)$

5. Time Complexity Chart

Power calculation: normal vs fast method

Theoretical number of recursive/iterative steps as the exponent increases.



The chart compares the growth of the normal and fast methods as the exponent increases. The **Normal $O(n)$** line rises quickly because its work grows roughly in proportion to n . The **Fast $O(\log n)$** line rises very slowly because the exponent is divided by 2 at every step. For $n = 256$, the normal method is on the order of 256 operations, whereas the fast method needs only about 8 halving levels. Therefore, fast exponentiation is much more efficient for large exponents.